ADCNC
Function Reference Manual
2.0

Wei-Li Chuang

Revision sheet:

Rev	Date	Author	Description
2.0	2014.10.14	Weili	Release ADCNC function reference manual

1. Introduction

The ADCNC library is a hardware-independent library which is developed for the demands of two-dimensional high speed and high precise machining in industry. Figure 1 illustrates the structure of ADLINK CNC system composed of ADCNC library, APS library and motion control card. The ADCNC library including blocks of read file, path optimization and velocity planning plays an important role in this CNC system. The read file block can import user's CAD/CAM file and the imported data can be processed by both path optimize and velocity planning or only velocity planning. The path optimize block is mainly responsible for line fitting of imported data. It has benefits of reducing machining time but the tradeoff is discontinuity of velocity and acceleration of each axis. The velocity planning block is used to generate smooth resultant velocity profile and acceleration profile for user's applications. A smooth velocity factor is employed to avoid vibration to improve manufacturing quality. The results of ADCNC library will transfer data to motion control card sequentially by APS library. Finally, the motion control card will generate smooth position commands to each axis to control the servo motor system by interpolation function for various applications.

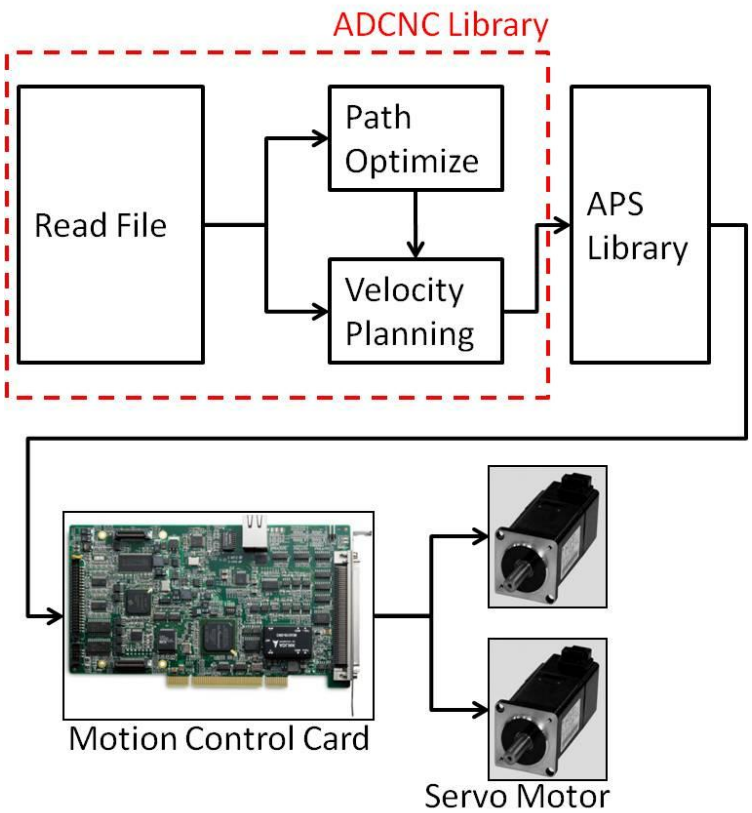


Figure 1 ADLINK CNC system

Now each block of ADCNC library will be introduced here.

(1) Read file block:

An example below shows a basic format of user's CAD/CAM file being able to recognize by read file block should include items *opt* (option), *x* and *y* (absolute position of axes or arc center) and *theta* (arc angle). The detailed bit define of *opt* will be introduced in the description of structure POS_DATA_2D_F64. On the other hand, the absolute position of each axis and arc angle can be floating values. The unit of arc angle is degree and its resolution is 10E-6. For example, if desired angle of arc is 90.0 degree, the input should be 90000000.0.

Example of file format for structure POS_DATA_2D_F64

```
//      opt          x            y           theta
0x00000308  0.0000000000   0.0000000000 0.0000000000
0x00000308 -0.0499995833   9.9998333342 0.0000000000
0x00000308 -0.1999933334  19.9986666933 0.0000000000
0x00000308 -0.4499662510  29.9955002025 0.0000000000
```

An advanced format for structure PNT_DATA_2D_F64 shown below is similar to that for structure POS_DATA_2D_F64. Besides *opt*, *x*, *y*, and *theta*, the additional terms *acc*, *dec*, *vi*, *vm*, and *ve* provide information of acceleration, deceleration, start velocity, maximum velocity, and end velocity respectively for velocity planning.

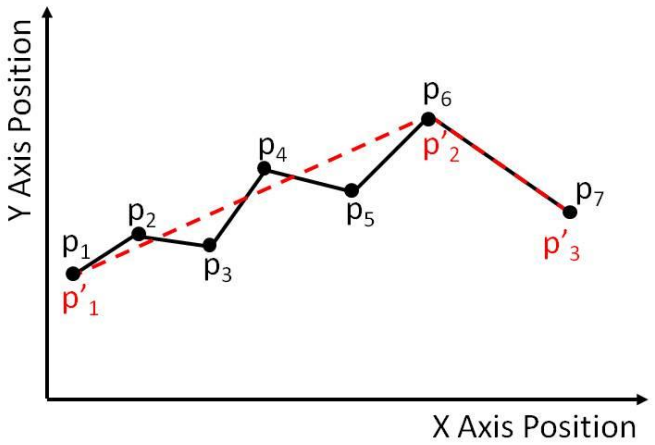
Example of file format for structure PNT_DATA_2D_F64

```
//    opt          x        y      theta    acc        dec        vi     vm         ve
0x00000308   4381.000  139.000  0.000  2000.000   2000.000   0.000  200.000   0.000
0x00000308   4381.000  154.000  0.000  1405.169   1056.169   0.000  997.638   0.000
0x00000308   4553.000  240.000  0.000  6535.395   1535.395   0.000  653.540   0.000
0x00000308   1129.000  340.000  0.000  2352.663   3112.663   0.000  992.999   0.000
```

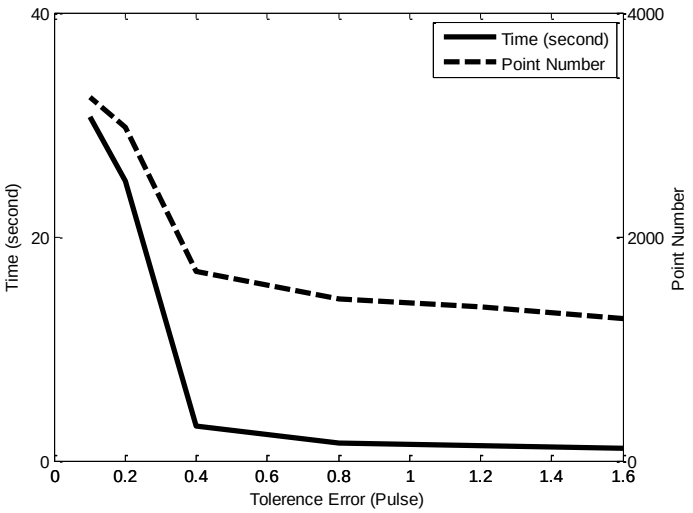
User should have basic understanding about how ADCNC library treat these imported points and data. Improper setting of these points will result in unexpected behavior easily. It is strongly recommended for user to test and run their own application by MCP2 first to avoid accident happening.

(2) Path optimize block:

Only one input parameter *tolerance error* is needed for path optimization, and its unit is pulse. Figure 2(a) shows the results of path optimization: Instead of original trajectory $p_1 \sim p_7$, this block will find a new trajectory $p_1' \sim p_3'$ automatically based on the *tolerance error* information. The length of new line $p_1' \sim p_2'$ is long enough to be capable to increase machining speed substantially. In figure 2(b), increasing *tolerance error* yields shorter time since point number decreases subcutaneously. If *tolerance error* is set zero, the path optimization block will find a default tolerance error by one-tenth minimum distance of this pattern and use it to generate new trajectory. One thing needed to know is the path optimization block will insert dwell points before and after rapid point to avoid distortion machining results. User has to take responsibility to choose *tolerance error* carefully to avoid issue of pattern distortion.



(a)



(b)

Figure 2 Results of path optimization

(3) Velocity planning block

In figure 3, the velocity planning block uses technique of connecting successive s-curve velocity profiles to reduce acceleration and deceleration time and improve machining efficiency. Unlike conventional trapezoidal velocity profile, the built-in cubic-order s-curve velocity profile guarantees to have continuous position, velocity, and acceleration during motion. The boundary velocity of velocity profiles is continuous and the start and end velocity of velocity profile are zero.

Figure 3 Results of velocity planning

A table shown below is to introduce the input parameters for velocity planning block. In order to have minimum motion time, velocity planning uses maximum acceleration/deceleration in ramping region and maximum velocity in cruise region to meet this requirement. The scenario of using max velocity/acceleration or rapid max velocity/acceleration for user depends on current motion itself is machining pieces or moving stage. One thing needs to know is start/end velocity keep zero when this point is set a rapid point. A velocity smooth factor is used to control jerk to reduce vibration when starting or breaking motor. The corner velocity limit is provided for user to adjust corner velocity. If this setting is zero, velocity planning will decrease corner velocity automatically. The input parameter cycle time is used for planning since the results of velocity planning must be integer times of it. If this setting is zero, a default value 1000 us will be used in this block. The cycle time parameter has to be equal to that used in motion control card.

Table of input parameters for velocity planning

Parameter	Description	Range	Unit
Maximum Velocity	Maximum velocity for machining	>0.0	Pulse per second
Maximum Acceleration	Maximum acceleration for machining	>0.0	Pulse per second squared

Rapid Maximum Velocity	Maximum velocity for moving	>0.0	Pulse per second
Rapid Maximum Acceleration	Maximum acceleration for moving	>0.0	Pulse per second squared
Smooth Velocity Factor	Adjust smoothness of velocity profile	0.0~1.0	Dimensionless
Corner Velocity Limit	Limit corner velocity	≥ 0; Suggested value is 8000000	Dimensionless
Cycle Time	Motion control time	≥ 0; if user sets 0, the default cycle time 1000us is used.	Nano second

In order to get well-planned velocity profile for machining, the following will try to make user have more understanding about the insight methods and theory of velocity planning.

a. Corner velocity constraint

In figure 4, in order to reduce effect of inertia, a corner deceleration mechanism is employed here, and the machining precision can be improved obviously. A description of calculating corner velocity V_{corner} is

$$V_{corner} \leq V_{max} \cdot exp^{(-\alpha\theta/1000000)}$$

Where V_{max} denotes maximum velocity of input parameter, α is corner velocity limit of input parameter and θ is the angle between lines or arc. It is suggested the value of α can be 8000000. This setting will make corner velocity be equal to about 1% maximum velocity when the angle of consequent lines or arc is larger than about 30 degree. If α is set zero, the V_{corner} will be obtained automatically by velocity planning

$$V_{corner} \leq \frac{Acc_{max}\Delta T}{2sin(\theta/2)}$$

Where Acc_{max} is maximum acceleration of input parameter and ΔT is the cycle time of input parameter.

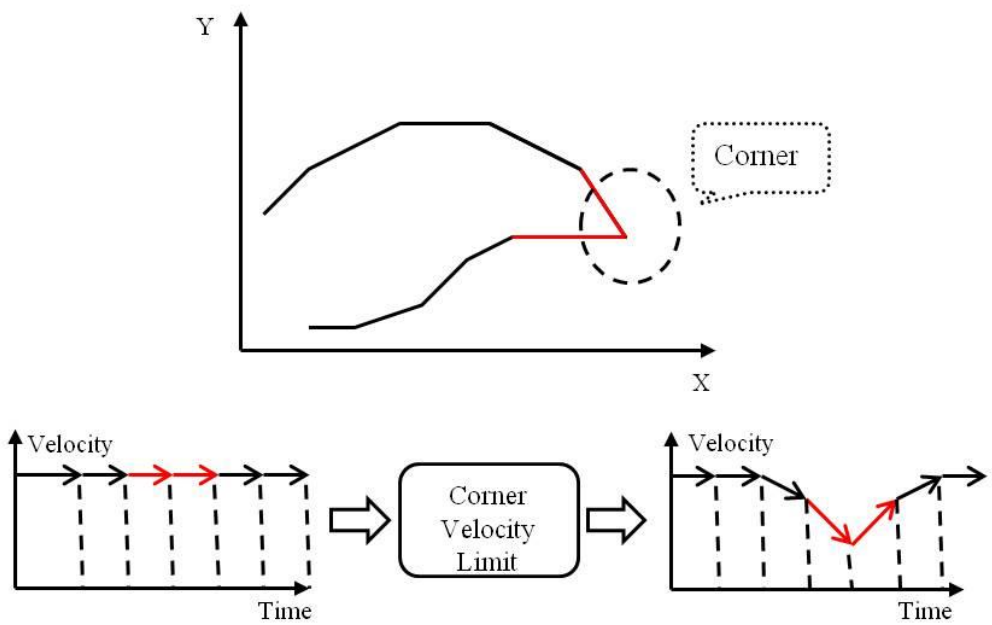


Figure 4 Corner velocity

b. Arc velocity constraint

The centripetal and tangential acceleration need to be taken into consideration when performing arc move function shown in figure 5. The maximum acceleration Acc_{arc} of arc is

$$Acc_{arc} \leq \frac{Acc_{max}}{\sqrt{2}}$$

Where the Acc_{max} is maximum acceleration of input parameter. On the other hand, the maximum velocity V_{arc} of arc can be found by

$$V_{arc} \leq \sqrt{r \cdot Acc_{arc}}$$

Where r denotes arc radius. The arc velocity will also have another upper limit of maximum velocity shown below

$$V_{arc} \leq V_{max}$$

Calculating of V_{arc} and Acc_{arc} are built-in algorithm in velocity planning block.

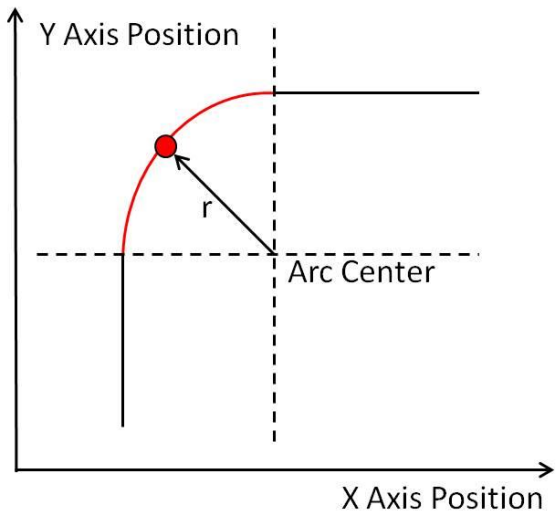


Figure 5 Arc move

c. Smooth velocity factor

Figure 6 shows results of various smooth velocity factor (*Sf* = 0.0 ~ 1.0). For the case of *Sf* being equal to 0.0, it forms a traditional trapezoidal velocity profile with shortest motion time but serious vibration. However, if *Sf* is set 1.0, it spends most time but has better vibration suppression. Usually, it is suggested for user to let smooth velocity factor be equal to 0.5 to have balance of machining efficiency and product quality.

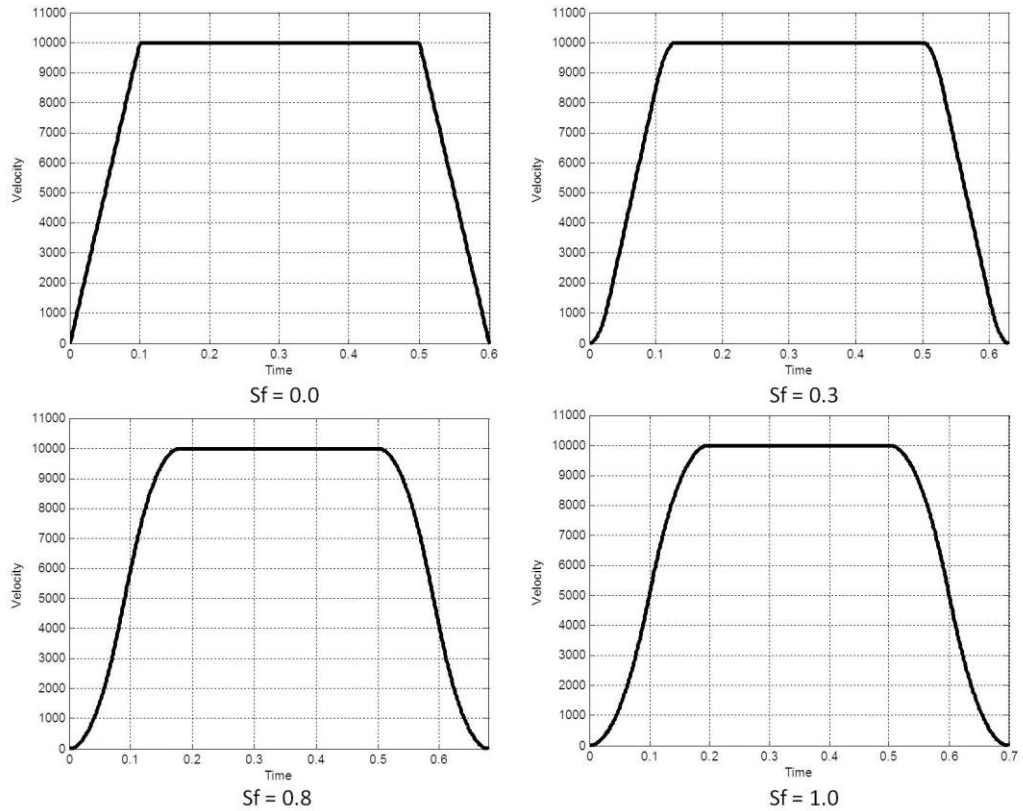
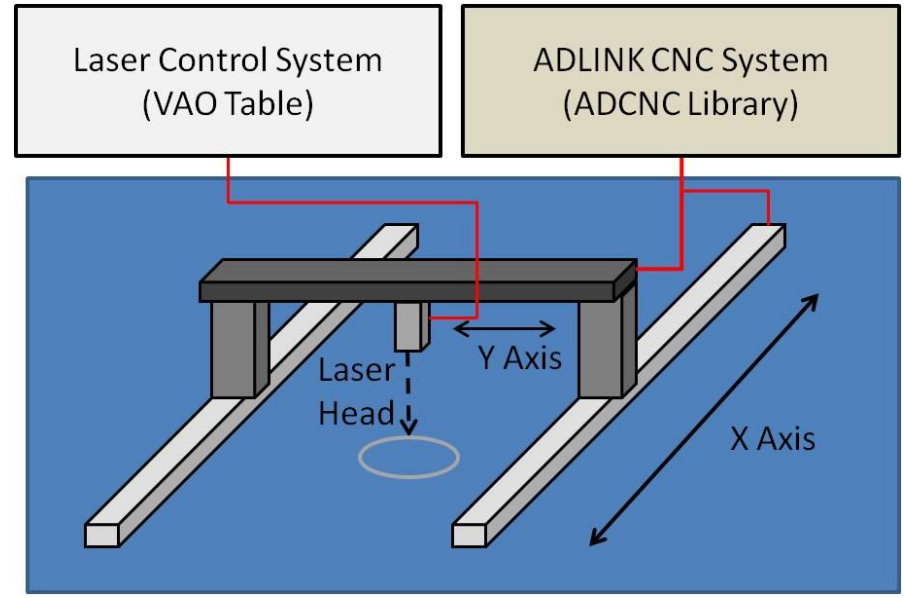


Figure 6 Velocity profiles using various smooth velocity factor *Sf*

(4) Application

The ADCNC library has strong capability to co-operate with other modules in ADLINK motion control card for various applications. In figure 7(a), an example of laser cutting application using laser control system and ADLINK CNC system is presented. The laser control system composed of VAO table and PWM generation module will calculate proper laser power by information of current command/feedback velocity and then generate PWM signal to the laser head. The ADLINK CNC system is responsible for controlling two axes (x and y). Figure 7 (b) shows the results of laser cutting by ADCNC library.



(a)

(b)

Figure 7 Application of ADLINK CNC system in laser cutting

2. List of all structures

Structure	Description
POS_DATA_2D_F64	2-dimension position data with F64 type
PNT_DATA_2D_F64	2-dimension point data with F64 type
VP_PARAM_CONFIG	velocity planning configuration

POS_DATA_2D_F64	2-dimension position data with F64 type

Descriptions

The POS_DATA_2D_F64 structure contains 2-dimension position data with F64 type. This structure is defined in the "type_def.h" header file of the ADCNC library.

Syntax

C/C++:

```
typedef struct _POS_DATA_2D_F64
{
    U32 u32_opt;        // option, [0x00000000, 0xFFFFFFFF]
    F64 f64_x;          // x-axis component (pulse)
    F64 f64_y;          // y-axis component (pulse)
    F64 f64_theta;      // x-y plane arc move angle (0.000001 degree), [-360000, 360000]
}
POS_DATA_2D_F64, *PPOS_DATA_2D_F64;
```

Members

u32_opt

A bit set specifies the option.

7	6	5	4	3	2	1	Bit : 0
Rapid	Dwell	Reserved		Power_En	Linear/Arc	Reserved	
15	14	13	12	11	10	9	8
Layer_No(0~7)			Do_Ch (0~7)			Do_OnOff	Do_En

Bit 0~1: It is reserved for future use and must be zero.

Bit 2 : 1: Arc move, 0: Linear move

Bit 3: 1: Enable power control (when it is enabled, the setting layer is effective in bit 13 ~ 15), 0: Disable

Bit 4~5: It is reserved for future use and must be zero.

Bit 6: 1: Dwell enable, used *f64_x* field to specify dwell times (unit: ms)

Bit 7: 1: Rapid positioning (using *rvmax* and *racc*). 0: Operation positioning (using *vmax* and *acc*)

Bit 8: 1: Enable Digital output, 0: Disable Digital output

Bit 9: 1: Set Digital output on (set to 1), 0: Set Digital output off (set to 0)

Bit 10~12: Digital output channel (0 ~ 7)

Bit 13~15: Select a layer number from 0 to 7. It is effective when Bit 3 is enabled.

f64_x

A number specifies the x-axis component of the target position.

f64_y

A number specifies the y-axis component of the target position.

f64_theta

A number specifies the angle of the arc move. This number should be nonzero when arc moves.

PNT_DATA_2D_F64	2-dimension point data with F64 type.

Descriptions

The POS_DATA_2D_F64 structure contains 2-dimension point data with F64 type. This structure is defined in the "type_def.h" header file of the ADCNC library.

Syntax

C/C++:

```
typedef struct _PNT_DATA_2D_F64
{
    U32 u32_opt;        // option, [0x00000000,0xFFFFFFFF]
    F64 f64_x;          // x-axis component (pulse), [-2147483648,2147484647]
    F64 f64_y;          // y-axis component (pulse), [-2147483648,2147484647]
    F64 f64_theta;      // x-y plane arc move angle (0.000001 degree), [-360000,360000]
    F64 f64_acc;        // acceleration rate (pulse/ss), [0,2147484647]
    F64 f64_dec;        // deceleration rate (pulse/ss), [0,2147484647]
    F64 f64_vi;         // initial velocity (pulse/s), [0,2147484647]
    F64 f64_vm;         // maximum velocity (pulse/s), [0,2147484647]
    F64 f64_ve;         // ending velocity (pulse/s), [0,2147484647]
    F64 f64_sf;         // s-factor [0.0 ~ 1.0]
}
PNT_DATA_2D_F64, *PPNT_DATA_2D_F64;
```

Members

u32_opt

A bit set specifies the option. The bit define is same with POS_DATA_2D_F64.

f64_x

A number specifies the x-axis component of the target position. This unit is pulse.

f64_y

A number specifies the y-axis component of the target position. This unit is pulse.

f64_theta

A number specifies the angle of the arc move. This number should be nonzero when arc moves.

f64_acc

A positive number specifies the acceleration rate. This unit is pulse/second^2.

f64_dec

A positive number specifies the deceleration rate. This unit is pulse/second^2.

f64_vi

A positive number specifies the initial velocity. This unit is pulse/second.

f64_vm

A positive number specifies the maximum velocity. This unit is pulse/second.

f64_ve

A positive number specifies the ending velocity. This unit is pulse/second.

f64_sf

Smooth velocity factor. Its range is from 0.0 to 1.0.

VP_PARAM_CONFIG	velocity planning configuration

Descriptions

The VP_PARAM_CONFIG structure contains configurations used for velocity planning. The first two members (*vmax* and *acc*) are necessary information for basic velocity planning. However, rapid move uses next two members (*rvmax* and *racc*). Other configurations such like corner velocity limit, cycle time, and smooth velocity factor can refer to the description below.

Syntax

C/C++:

```
typedef struct _VP_PARAM_CONFIG
{
   U32     vmax;       // maximum velocity (pulse/s)
   U32     acc;        // maximum acceleration (pulse/s^2)
   U32     rvmax;      // rapid, maximum velocity (pulse/s)
   U32     racc;       // rapid, maximum acceleration (pulse/s^2)
   U32     corner;     // corner velocity setting
   U32     dt;         // cycle time (unit: nano second)
   U32     opt;        // reserved
   F64     sfactor;    // s-curve factor
}
VP_CONFIG, *PVP_CONFIG;
```

Members

vmax

A positive number specifies the maximum velocity. The unit is pulse/second. This number should always be nonzero.

acc

A positive number specifies the acceleration. This number must be nonzero.

rvmax

Rapid move configuration for maximum velocity (pulse/s); this number must be nonzero.

racc

Rapid move configuration for acceleration rate (pulse/s^2); this number must be nonzero.

corner

Corner velocity limit setting (the suggested value is 8000000)

dt

Cycle time of motion control card (nano-second); it will use 1000 us if this setting is zero.

sfactor

S-curve factor; its range is from 0.0 to 1.0

Opt

It is reserved for future use

3. List of all functions

Function	Description
ADCNC_Version	This function will get the current version number of this library
ADCNC_Path_Optimize	This function will reorganize the data point and find a suitable representation for original pattern.
ADCNC_Velocity_Planning	This velocity planning function will generate smooth velocity profile
PosData2_FileLoad_F64	This function will load POS_DATA_2D_F64 information from file
PosData2_FileSave_F64	This function will save POS_DATA_2D_F64 information into file
PntData2_FileLoad_F64	This function will load PNT_DATA_2D_F64 information from file
PntData2_FileSave_F64	This function will save PNT_DATA_2D_F64 information into file

ADCNC_Version	to get the current version number of this package

Support Products

Descriptions

This function is used to get the current version number of ADCNC library. Please check it before starting your utility.

Syntax

C/C++:

```
I32 __stdcall
ADCNC_Version(void);
```

Parameters

This function has no parameter.

Return Values

The data format of the return value is “yyyymmddnn”, e.g. 2010051701.

Example Code

```
I32 ver = 0;
ver = ADCNC_Version();
```

ADCNC_Path_Optimize	to reorganize the data point using line fitting technique

Support Products

Descriptions

According to the input tolerance number and the position data, this function is capable to remove the redundant position data. Be careful that the positive number "*reducedpointnumber*" may be smaller, equal to, or larger than "*pointnumber*". In default, when the "*ToleranceError*" was set as 0, the "*df_tolerr*" will return a number computed by the 1/10 of the minimum distance in pattern.

Syntax

C/C++:

```
I32 __stdcall
ADCNC_Path_Optimize(
        U32 pointnumber                 // [IN] : position array counter
      , POS_DATA_2D_F64* PosArrayIn     // [IN] : position array start pointer
      , F64 ToleranceError              // [IN] : tolerance error
      , U32* reducepointnumber          // [OUT] : position array counter after reduction
      , POS_DATA_2D_F64* PosArrayOut    // [OUT] : position array start pointer after reduction
      , F64* df_tolerr // [OUT] : default tolerance error (1/10 minimun distance of this pattern)
      );
```

Parameters

pointnumber

[in] A positive number specifies the count of the position data.

PosArrayIn

[in] A pointer indicates the starting address of the position data.

ToleranceError

[in] A positive number specifies the tolerance count to smooth.

reducedpointnumber

[out] A pointer points to a positive number which specifies the output count of the position data after path optimization.

PosArrayOut

[out] A pointer indicates the starting address of the output point data after optimization. A block of memory is filled by this function, which is started at this pointer and counted by "PosCountO".

Df_tolerr

[out] A pointer points to a positive number when the "ToleranceError" was set as 0.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 pointnumber  = 0;           // Point number of file
U32 reducepointnumber = 0;      // Output point number of path optimize
POS_DATA_2D_F64 *PosArrayIn;    // Input of path optimize
POS_DATA_2D_F64 *PosArrayOut;   // Output of path optimize
F64 ToleranceError = 1;         // Tolerence error of path optimize
F64 df_tolerror = 0;            // Default tolerence error of path optimize
static const wchar_t  __filei[] = L".\\circle1.txt"; // File location

// Get data size for PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, 0 );

// Allocate memory for PosArrayIn
PosArrayIn = (POS_DATA_2D_F64*) calloc(pointnumber, sizeof(POS_DATA_2D_F64));

// Read data into PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, PosArrayIn );

// Get data size for PosArrayOut
ret = ADCNC_Path_Optimize( pointnumber, PosArrayIn, ToleranceError, &reducepointnumber, 0,
&df_tolerror );

// Allocate memory for PosArrayOut
PosArrayOut  = (POS_DATA_2D_F64*)calloc( reducepointnumber, sizeof(POS_DATA_2D_F64) );

// Start path optimization algorithm
ret = ADCNC_Path_Optimize( pointnumber, PosArrayIn, ToleranceError, &reducepointnumber,
PosArrayOut, &df_tolerror );
```

ADCNC_Velocity_Planning	velocity planning on a standalone trajectory with F64 data type input

Support Products

Descriptions

According to the input configuration and the position data, this function is capable to generate smooth velocity data in the 2-D point data (PNT_DATA_2D_F64) array with information of velocity, acceleration and so on for interpolation function. The initial velocity of the first point and the finish velocity of last point must be zero. The output of velocity planning function can be transferred to motion control card by the APS library.

Syntax

C/C++:

```
I32 __stdcall
ADCNC_Velocity_Planning(
      VP_PARAM_CONFIG* VpConfig  // [IN ] velocity planning configuration, [!NULL]
    , U32 PosCount               // [IN ] position array count, [1,4294967295]
    , POS_DATA_2D_F64* PosArray  // [IN ] position array start pointer, [!NULL]
    , PNT_DATA_2D_F64* PntArray  // [OUT] point array start pointer, [!NULL]
    );
```

Parameters

VpConfig

[in] A pointer points to the VP_CONFIG structure that specifies the velocity planning configuration.

PosCount

[in] A positive number specifies the count of the position data to plan. This value must be nonzero.

PosArray

[in] A pointer indicates the starting address of the position data to plan.

PntArray

[out] A pointer indicates the starting address of the point data to plan. A block of memory is filled by this function, which is started at this pointer and counted by “PosCount”.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 pointnumber  = 0;           // Point number of file
U32 reducepointnumber = 0;      // Output point number of path optimize
POS_DATA_2D_F64 *PosArrayIn;    // Input of path optimize
POS_DATA_2D_F64 *PosArrayOut;   // Output of path optimize
F64 ToleranceError = 1;         // Tolerence error of path optimize
F64 df_tolerror = 0;            // Default tolerence error of path optimize
PNT_DATA_2D_F64 *PntArrayOut;   // Output of velocity planning
VP_PARAM_CONFIG VpConfig;       // Configuration for velocity planning
static const wchar_t  __filei[] = L".\\circle1.txt"; // File location

// Get data size for PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, 0 );

// Allocate memory for PosArrayIn
PosArrayIn = (POS_DATA_2D_F64*) calloc(pointnumber, sizeof(POS_DATA_2D_F64));

// Read data into PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, PosArrayIn );

// Get data size for PosArrayOut
ret = ADCNC_Path_Optimize( pointnumber, PosArrayIn, ToleranceError, &reducepointnumber, 0,
&df_tolerror );

// Allocate memory for PosArrayOut
PosArrayOut  = (POS_DATA_2D_F64*)calloc( reducepointnumber, sizeof(POS_DATA_2D_F64) );

// Start path optimization algorithm
ret = ADCNC_Path_Optimize( pointnumber, PosArrayIn, ToleranceError, &reducepointnumber,
PosArrayOut, &df_tolerror );

// Allocate memory for PntArrayOut
PntArrayOut = (PNT_DATA_2D_F64*)calloc( reducepointnumber,  sizeof(PNT_DATA_2D_F64) );

// Velocity planning configuration
VpConfig.vmax    = 1000000;  // maximum velocity (pulse/s), [1,4294967295]
VpConfig.acc     = 10000000; // acceleration rate (pulse/ss), [1,4294967295]
VpConfig.rvmax   = 1500000;  // rapid, maximum velocity (pulse/s), [1,4294967295]
VpConfig.racc    = 20000000; // rapid, acceleration rate (pulse/ss), [1,4294967295]
VpConfig.sfactor = 0.5;      // smooth velocty factor
VpConfig.corner  = 8000000;  // corner velocity limit
VpConfig.dt      = 1000000;  // cycle time

// Start velocity planning algorithm
ret = ADCNC_Velocity_Planning( &VpConfig, reducepointnumber, PosArrayOut, PntArrayOut );
```

PosData2_FileLoad_F64	to load POS_DATA_2D_F64 information from file

Support Products

Descriptions

This function is used to load POS_DATA_2D_F64 information from file.

Syntax

C/C++:

```
I32 __stdcall
PosData2_FileLoad_F64(
      const wchar_t* FileName   // [IN ] file name
    , U32* PosCount             // [OUT] array count
    , POS_DATA_2D_F64* PosArray // [OUT] array start pointer, position data 2d
    );
```

Parameters

FileName

[in] A pointer points to a literal string that specifies the name of the file.

PosCount

[out] A positive number specifies the count of total position data in the file.

PosArray

[out] A pointer indicates the starting address of the position data to be stored.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 pointnumber  = 0;           // Point number of file
U32 reducepointnumber = 0;      // Output point number of path optimize
POS_DATA_2D_F64 *PosArrayIn;    // Input of path optimize
static const wchar_t  __filei[] = L".\\circle1.txt"; // File location

// Get data size for PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, 0 );

// Allocate memory for PosArrayIn
PosArrayIn = (POS_DATA_2D_F64*) calloc(pointnumber, sizeof(POS_DATA_2D_F64));

// Read data into PosArrayIn
ret = PosData2_FileLoad_F64( __filei, &pointnumber, PosArrayIn );
```

PosData2_FileSave_F64	to save POS_DATA_2D_F64 information into file

Support Products

Descriptions

This function is used to save POS_DATA_2D_F64 information into file.

Syntax

C/C++:

```
I32 __stdcall
PosData2_FileSave_F64(
      const wchar_t* FileName   // [IN ] file name
    , U32 PosCount              // [IN ] array count
    , POS_DATA_2D_F64* PosArray // [IN ] array start pointer, position data 2d
    );
```

Parameters

FileName

[in] A pointer points to a literal string that specifies the name of the file.

PosCount

[in] A positive number specifies the count of total position data.

PosArray

[in] A pointer indicates the starting address of the position data to save.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 reducepointnumber = 0;      // Output point number of path optimize
POS_DATA_2D_F64 *PosArrayOut;   // Output of path optimize
static const wchar_t  __fileo[] = L".\\output.txt"; // File location

// Process of getting reducepointnumber and saving data to structure PosArrayOut...

// Save data into file
ret = PosData2_FileSave_F64( __fileo, reducepointnumber, PosArrayOut );
```

PntData2_FileLoad_F64	to load PNT_DATA_2D_F64 information from file

Support Products

Descriptions

This function is used to load PNT_DATA_2D_F64 information from file.

Syntax

C/C++:

```
I32 __stdcall
PntData2_FileLoad_F64(
      const wchar_t* FileName      // [IN ] file name
    , U32* PntCount                // [OUT] array count
    , PNT_DATA_2D_F64*  PntArray   // [OUT] array start pointer, point data 2d
    );
```

Parameters

FileName

[in] A pointer points to a literal string that specifies the name of the file.

PntCount

[out] A positive number specifies the count of total point data in the file.

PntArray

[out] A pointer indicates the starting address of the point data to be stored.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 pointnumber  = 0;           // File data size
PNT_DATA_2D_F64 *PntArray;      // Structure to load data
static const wchar_t  __filei[] = L".\\dragon.txt"; // File location

// Check data size
ret = PntData2_FileLoad_F64( __filei, &pointnumber, 0 );

// Allocate memory
PntArray = (PNT_DATA_2D_F64*) calloc(pointnumber, sizeof(PNT_DATA_2D_F64));

// Read data into PntArray
ret = PntData2_FileLoad_F64( __filei, &pointnumber, PntArray );
```

PntData2_FileSave_F64	to save PNT_DATA_2D_F64 information into file

Support Products

Descriptions

This function is used to save PNT_DATA_2D_F64 information into file.

Syntax

C/C++:

```
I32 __stdcall
PntData2_FileSave_F64(
      const wchar_t* FileName      // [IN ] file name
    , U32 PntCount                 // [IN ] array count
    , PNT_DATA_2D_F64* PntArray    // [IN ] array start pointer, point data 2d
    );
```

Parameters

FileName

[in] A pointer points to a literal string that specifies the name of the file.

PntCount

[in] A positive number specifies the count of total point data.

PntArray

[in] A pointer indicates the starting address of the point data to save.

Return Values

Please refer to error code table.

Example Code

```
#include "ADCNC2.h"
I32 ret = 0;                    // Return value
U32 reducepointnumber = 0;
PNT_DATA_2D_F64 *PntArray;
static const wchar_t  __fileo[] = L".\\output.txt"; // File location

// Process of getting reducepointnumber and saving data to structure PntArray...

// Save data into file
ret = PntData2_FileSave_F64 ( __fileo, reducepointnumber, PntArray);
```

4. Error code table

Value	Name	Descriptions
0	ERR_CNC_NOERR	success / no error
-10001	ERR_CNC_INVALID_ARGUMENT	invalid argument
-10002	ERR_CNC_INVALID_CONFIG	invalid configuration
-10006	ERR_CNC_FILING_FAILED	file operation is illegal or failed